

Лекция 4: Конструкции алгоритмов: линейная и ветвление

План:

- 1) Конструкция линейного алгоритма
- 2) Конструкция алгоритма ветвления

Различают следующие алгоритмические конструкции: линейные, ветвления и цикл.

Линейная алгоритмическая конструкция

Линейной называют алгоритмическую конструкцию, реализованную в виде последовательности действий (шагов), в которой каждое действие (шаг) алгоритма выполняется ровно один раз, причем после каждого i -го действия (шага) выполняется $(i+1)$ -е действие (шаг) если i -е действие не конец алгоритма (рис.1).

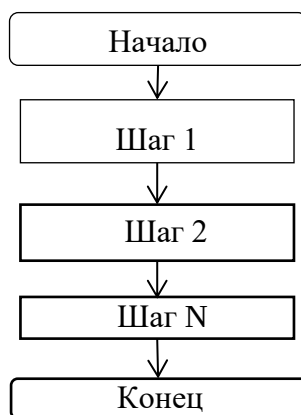


Рисунок 1 - Линейная блок-схема

Пример линейного алгоритма: пользователь вводит два числа. Программа вычисляет их сумму и выводит результат на экран.

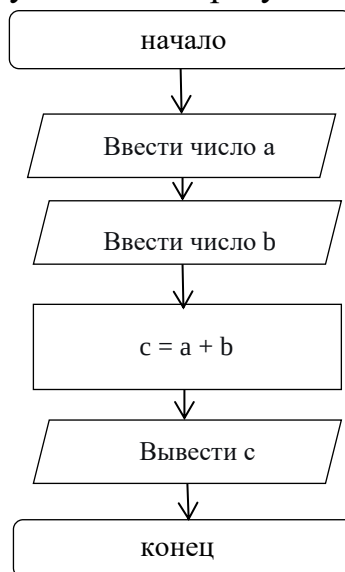


Рисунок 2 – Блок-схема задачи

Словесное описание:

Запросить у пользователя первое число.

Запросить у пользователя второе число.
Сложить эти два числа.
Вывести результат на экран.

Program zadanie1;

var

a, b, sum: real; // Используем real для поддержки дробных чисел

begin

// 1. Запросить у пользователя первое число

write('Введите первое число: ');

readln(a);

// 2. Запросить у пользователя второе число

write('Введите второе число: ');

readln(b);

// 3. Сложить эти два числа

sum := a + b;

// 4. Вывести результат на экран

writeln('Сумма чисел ', a, ' и ', b, ' равна ', sum);

readln; // Ожидание нажатия Enter перед закрытием программы
end.

КОНСТРУКЦИЯ ВЕТВЛЕНИЯ

Конструкцией ветвления называют конструкцию, которая обеспечивает выбор между двумя альтернативами в зависимости от значения входных данных. При каждом намеренном выборе входных данных алгоритм ветвления сводится к линейному алгоритму. Различают неполное (если – то) и полное (если – то – иначе), каждая из которых ведет к общей точке их слияния, каждое выполнение алгоритма продолжается независимо от того, какой путь был выбран (рис. 3).

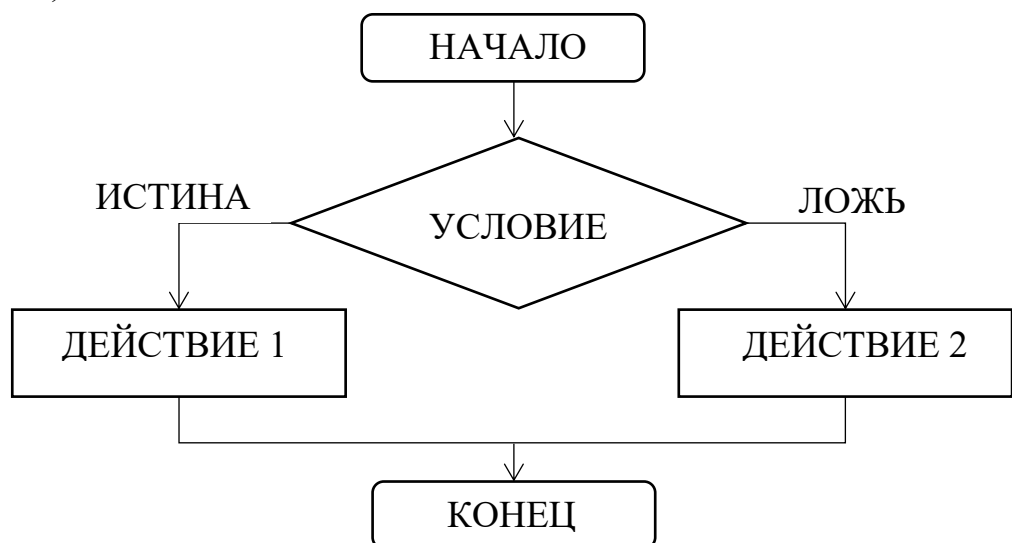


Рисунок 3 – Блок – схема ветвления

Условный оператор IF

Оператор If реализует алгоритмическую конструкцию «Решение» и позволяет изменить порядок выполнения операторов в зависимости от истинности или ложности некоторого условия. Формат оператора может быть записан в двух формах: полной и неполной форме.

Формат оператора в полной форме:

If<условие> Then<оператор 1> Else<оператор 2>

Формат оператора в неполной форме:

If<условие>Then<оператор 1>

If (если), Then (тогда), Else (иначе) – зарезервированные слова.

Условный оператор работает следующим образом. Сначала вычисляется условное выражение < условие >. Если результат есть True (истина), то выполняется < оператор 1 > (после слова Then), а < оператор 2 > пропускается. Если результат есть False (ложь), то < оператор 1 > пропускается, а выполняется < оператор 2 > (после слова Else). Перед Else никогда не ставится точка с запятой, поскольку это еще не конец оператора If.

Часть Else <оператор 2> условного оператора If может быть опущена. Если < условие > оператора If принимает значение True, то выполняется <оператор 1>, в противном случае этот оператор пропускается. В этом случае формат условного оператора имеет неполную форму:

Сравним две конструкции:

If <условие>Then<оператор 1>;

If <условие>Then<оператор 1>; <оператор 2>;

В первой строке <оператор 1> или выполняется, или не выполняется в зависимости от выполнения <условия>. Во второй строке <оператор 2> выполняется всегда, поскольку после <оператора 1> стоит разделитель «;» и все что стоит после разделителя считается другим оператором. Поэтому в соответствии с форматом оператора If после зарезервированных слов Then и Else может стоять всего лишь один оператор.

Пример:

If a > 0 Then b:=1;

If a > 0 Then b:=1; a:=b; {оператор a:=b; выполняется всегда}

Если же необходимо выполнить после Then и Else не один, а несколько операторов, то следует поставить операторные скобки, т.е. использовать составной оператор.

В этом случае оператор If будет иметь следующий вид:

```
If<условие>Then
```

```
begin
```

```
оператор 11;
```

```
...
```

```
оператор 1n;
```

```
end
```

```
Else
```

```
begin
```

```
оператор 21;
```

```
...
```

```
оператор 2n;
```

```
end;
```

Типичные ошибки, которые совершают при написании программного кода, представлены на таблице 2.

Таблица 1 – ошибки

Ошибка	Неправильно	Правильно
Точка с запятой перед Else	if a>b then a:=1; else a:=2;	if a>b then a:=1 else a:=2;
Присваивание вместо сравнения	if a := 1 then writeln('Один');	if a = 1 then writeln('Один');
Отсутствие begin..end для нескольких операторов	if a>b then a:=1; b:=2;	if a>b then begin a:=1; b:=2; end;
Неправильная запись двойного неравенства	if (1 < x < 10) then ...	if (x > 1) and (x < 10) then ...
Точка с запятой после Then перед Else	if a>b then a:=1; else a:=2;	if a>b then a:=1 else a:=2;
Использование Then в Else ветке	if a>b then a:=1 else then b:=2;	if a>b then a:=1 else b:=2;
Сравнение вещественных чисел на равенство	if x = 0.1 then ...	if abs(x - 0.1) < 0.00001 then ...
Использование Real в операторе Case	case x of ... end; (где x: real)	Использовать if вместо case
Несовпадение типов в Case	case day of 1: ... 'A': ... (смешаны типы)	Все значения одного типа с выражением
Пропуск End в операторе Case	case day of 1: writeln('Пн'); 2: writeln('Вт');	case day of 1: writeln('Пн'); 2: writeln('Вт'); end;
Использование переменной вместо константы в Case	case day of x: writeln('Найдено');	case day of 1: writeln('Найдено');
Отсутствие пробелов между служебными словами	if a>bthen b:=1;	if a>b then b:=1;
Путаница между логическими операциями And/Or	if (x > 0) & (x < 10) then ...	if (x > 0) and (x < 10) then ...
Неправильное использование Else без полной формы	if a>b then a:=1; else; b:=2;	if a>b then a:=1 else b:=2;
Забыли двоеточие после значения в Case	case day of 1 writeln('Пн'); end;	case day of 1: writeln('Пн'); end;

Вложенный If без begin..end при Else	if a>0 then if b>0 then c:=1 else c:=2;	if a>0 then begin if b>0 then c:=1 end else c:=2;
Использование ; после Then в неполной форме	if a>b then; a:=1;	if a>b then a:=1;
Забыли Else в полной форме	if a>b then a:=1 (без else и без ;)	if a>b then a:=1 else a:=2;
Использование = вместо := в присваивании	if a = 1 then a = 2;	if a = 1 then a := 2;
Путаница между And и Or	if (x > 0) or (x < 10) then ... (нужно and)	if (x > 0) and (x < 10) then ...

Оператор выбора Case

Когда нужно выбрать одно действие из многих вариантов в зависимости от значения переменной, использовать цепочку if...else if...else неудобно. Оператор Case делает код чище и понятнее.

CASE выражение OF

значение1 : оператор1;

значение2 : оператор2;

значение3 : оператор3;

...

ELSE операторN;

END;

Где:

выражение – выражение порядкового типа (целое, символ, логический, перечислимый)

значение – константа того же типа, что и выражение

оператор – один или несколько операторов (составной через begin..end)

ELSE – выполняется, если ни одно значение не подошло (необязательная часть)

Важно: В Case нельзя использовать вещественные числа (Real)!

Простые примеры

Пример 1: Номер дня недели

pascal

program DayOfWeek;

var

day: integer;

begin

Write('Введите номер дня (1-7): ');

ReadLn(day);

case day of

1: WriteLn('Понедельник');

```

2: WriteLn('Вторник');
3: WriteLn('Среда');
4: WriteLn('Четверг');
5: WriteLn('Пятница');
6: WriteLn('Суббота');
7: WriteLn('Воскресенье');
else
  WriteLn('Ошибка: нет такого дня недели');
end;

```

```

  ReadLn;
end.

```

Пример 2: Оценка по букве

pascal

```

program Grade;

```

```

var

```

```

  letter: char;

```

```

begin

```

```

  Write('Введите оценку (A, B, C, D, F): ');

```

```

  ReadLn(letter);

```

```

  case letter of

```

```

    'A': WriteLn('Отлично (90-100)');

```

```

    'B': WriteLn('Хорошо (80-89)');

```

```

    'C': WriteLn('Удовлетворительно (70-79)');

```

```

    'D': WriteLn('Посредственно (60-69)');

```

```

    'F': WriteLn('Неудовлетворительно (0-59)');

```

```

  else

```

```

    WriteLn('Неизвестная оценка');

```

```

  end;

```

```

  ReadLn;

```

```

end.

```

В данной лекции рассмотрены линейный алгоритм и алгоритм ветвления. Изучены полная и неполная формы условного оператора If, а также правила использования операторных скобок begin..end. Рассмотрены типичные ошибки и представлен оператор выбора Case для множественного ветвления. Полученные знания позволяют создавать программы с разветвляющейся логикой.

Вопросы для проверки:

Вопрос 1 – Что такое линейный алгоритм? Приведите пример.

Вопрос 2 – Чем отличается полная форма условного оператора If от неполной?

Вопрос 3 – Для чего нужны операторные скобки begin..end в конструкции If?

Вопрос 4 – Какой оператор удобно использовать, когда нужно выбрать одно действие из многих вариантов (например, дни недели)? Приведите пример.